Tipi algebrici e alberi

L05 · Programmazione funzionale · OCaml

Mauro Leoncini

Obiettivi

- Somma e prodotto: perché si dice che sono «algebrici»
- `option`: la risposta al debito lasciato aperto a L03
- Rendere **non rappresentabili** gli stati illegali
- Alberi, e alberi le cui foglie sono numeri e i cui nodi sono operazioni

Il debito di L03

```
piu_lunga : string list -> string
```

Sulla lista vuota la firma promette una stringa e non c'è niente da restituire. Tre risposte non adeguate, che conosciamo:

1. sollevare un'eccezione — il chiamante può ignorarla
2. restituire `""` — un valore legittimo confuso con un errore
3. restituire `none` — ma come si fa, dal punto di vista della tipizzazione di una funzione, a restituire qualche volta una stringa e qualche altra volta `none`?

La risposta, ed è banale

```
type 'a option =
  | None
  | Some of 'a
```

Non è un costrutto del linguaggio, non è una parola chiave, non c'è niente di speciale: è **un tipo somma ordinario**, che potremmo definire noi, come abbiamo definito `lista` in L04.

```
piu_lunga : string list -> string option
```

Adesso la firma dice è pienamente informativa.

Il chiamante non può dimenticarsene

```
match piu_lunga l with
| None -> "la lista è vuota"
| Some s -> "la più lunga è: " ^ s
```

Il tipo **obbliga** ad indagare sul risultato, e l'esaustività di L04 verifica che entrambi i casi siano trattati.

La differenza con `null` non è filosofica: il compilatore vi ferma!

–>

Perché «algebrici»: si contano

Un tipo è un insieme di valori. Contiamoli.

```
type coppia = bool * bool                          (* 2 · 2 = 4 *)

type scelta = Sinistra of bool | Destra of bool    (* 2 + 2 = 4 *)
```

- `coppia` tiene **insieme** due `bool`: i valori si **moltiplicano**
- `scelta` è **un'alternativa** fra due `bool`: i valori si **sommano**

Da qui i nomi: tipo **prodotto** e tipo **somma**. Quelli che a L04 abbiamo chiamato «tipi definiti per casi» sono tipi somma. E in particolare:

$$|\texttt{'a option}| \;=\; 1 + |\texttt{'a}|$$

`unit option` ha due valori: `None` e `Some ()`. È `bool` con altri nomi.

L'etichetta fa la somma

In `Sinistra of bool | Destra of bool` entrambe le alternative contengono un `bool`.

- come **unione di insiemi**, `bool` ∪ `bool` è ancora `bool`: 2 valori
- come **tipo somma** i valori sono 4: `Sinistra true` e `Destra true` sono **diversi**

Il costruttore fa da **etichetta**: ricorda da quale alternativa viene il valore. È per questo che il `match` può sempre rispondere alla domanda «quale alternativa è?».

In C++ li conoscete già

```
struct Coppia { bool a; bool b; };                 // prodotto: 2 · 2
union  Nuda   { bool s; bool d; };                 // né l'uno né l'altra
using  Scelta = std::variant<Sinistra, Destra>;    // somma: 2 + 2
```

- `struct` è un **prodotto**
- `union` **non** è una somma: nessuno ricorda quale campo è attivo, e leggere quello sbagliato è comportamento indefinito
- `std::variant` **è** una somma: una `union` con l'etichetta, che si legge con `index()`

Le liste di L04, contate

```
type 'a list = [] | (::) of 'a * 'a list
```

Una **somma** di due alternative: `[]`, che ha un solo valore, **oppure** un **prodotto** di un elemento e di una lista. In formula:

$$L \;=\; 1 + A \times L$$

Sostituendo L con la sua definizione, ancora e ancora:

$$L = 1 + A + A^2 + A^3 + \cdots$$

La lista vuota, **oppure** un elemento, **oppure** due, **oppure** tre…: l'addendo A^n sono le liste di lunghezza n.

La domanda che cambia il mestiere

Non «quali valori il mio programma deve accettare?»

> **Quali valori il mio tipo permette di scrivere?**

Ogni valore che il tipo permette e la specifica vieta diventa un controllo a tempo di esecuzione, un'eccezione, un commento, e prima o poi un difetto.

Uno stato illegale, rappresentabile

Specifica: *un contatto deve avere almeno uno fra email e telefono.*

```
type contact_info = {
  email : string option;
  phone : string option;
}
```

```
{ email = None; phone = None }
```

Compila benissimo. La specifica lo vieta, il tipo no — e ogni funzione che riceve un `contatto` deve difendersi da un caso che non doveva esistere.

Lo stesso vincolo, nel tipo

```
type contact_info =
  | Email of string
  | Phone of string
  | Both of string * string
```

Il caso vietato **non è scrivibile**.

Niente controllo a tempo di esecuzione, niente eccezione, niente da ricordarsi: è sparito dal linguaggio.

> La specifica è passata dal commento al tipo, e il tipo lo verifica il compilatore.

Alberi: la stessa ricetta delle liste, con un ramo in più

```
type 'a albero =
  | Foglia
  | Nodo of 'a albero * 'a * 'a albero
```

A L04: una lista è vuota, **oppure** un elemento davanti a una lista. Oggi: un albero è una foglia, **oppure** un nodo con **due** sottoalberi.

```
let rec height = function
  | Leaf -> 0
  | Node (l, _, r) -> 1 + max (height l) (height r)
```

Il tipo ricorre due volte, quindi ricorre due volte anche il codice.

Uno statement che rivedremo tre volte

Dimensione e visita in ordine di un albero

```
let rec size = function
  | Leaf -> 0
  | Node (l, _, r) -> size l + 1 + size r
let rec inorder = function
  | Leaf -> []
  | Node (l, x, r) -> append (append (inorder l) [ x ])  (inorder r)
```

Teorema:

```
dimensione t  =  List.length (in_ordine t)        per ogni t
```

- oggi lo **verifichiamo** su un esempio
- in **L08** lo dimostriamo per ogni albero, e la dimostrazione avrà la stessa forma: un caso per costruttore
- in **L09** lo facciamo verificare a QCheck su migliaia di alberi casuali

Quando l'albero è il programma

```
type espr =
  | Num of int
  | Somma of espr * espr
  | Prod of espr * espr
  | Opposto of espr

let rec valuta = function
  | Num n -> n
  | Somma (a, b) -> valuta a + valuta b
  | Prod (a, b) -> valuta a * valuta b
  | Opposto a -> - (valuta a)
```

Un **programma** è diventato un dato e la funzione `valuta` il suo **interprete**.

Il tipo come lista di cose da fare

Aggiungete un costruttore:

```
  | Sottrazione of espr * espr
```

Compilate **senza toccare nient'altro**. Il compilatore vi elenca una per una tutte le funzioni da aggiornare: `valuta`, `stampa`, `profondita`.

> Un tipo non è documentazione. È una lista di cose da fare che "qualcuno" tiene aggiornata al posto vostro.

Lo stesso tipo, in C++

```
struct Num     { int n; };
struct Somma   { P a, b; };
struct Prod    { P a, b; };
struct Opposto { P a; };

struct Espr {
    std::variant<Num, Somma, Prod, Opposto> v;
};
```

`std::variant` è il tipo somma: «una di queste quattro cose, e si sa quale». Nome diverso, idea identica.

Due cose che in C++ si pagano

```
// using P = std::shared_ptr<const Espr>;
using P = Expr*;

//P num(int n) { return std::make_shared<const Espr>(Espr{Num{n}}); }
P num(int n)   { return new Expr{Num{n}}; }
```

- la **ricorsione**: un `variant` deve conoscere la dimensione delle sue alternative, quindi il tipo non può contenere sé stesso — serve un'indirezione esplicita
- i **costruttori**: in OCaml `Num 3` è già un valore

Il pattern matching, con quello che c'è

```
template <class... F> struct casi : F... { using F::operator()...; };

int valuta(const P& e) {
    return std::visit(casi{
        [](const Num& x)     { return x.n; },
        [](const Somma& x)   { return valuta(x.a) + valuta(x.b); },
        [](const Prod& x)    { return valuta(x.a) * valuta(x.b); },
        [](const Opposto& x) { return -valuta(x.a); },
    }, e->v);
}
```

Un caso per alternativa. È la ricetta di L04, scritta con gli strumenti del linguaggio.

E il controllo di esaustività c'è

Togliendo il caso `Opposto`, il programma **non compila**.

```
$ g++ -std=c++20 -c espressioni.cpp 2>&1 | wc -l
144
```

Centoquarantaquattro righe, quasi tutte dentro `<variant>`, in codice che non abbiamo scritto noi.

Dove sta l'informazione

Il primo errore, alla riga 25 di 144, nomina davvero il caso mancante:

```
error: no type named 'type' in 'struct std::invoke_result<
cases<valuta(const P&)::<lambda(const Num&)>, ...>, const Neg&>'
```

Contro OCaml, a L04:

```
Warning 8 [partial-match]: this pattern-matching is not exhaustive.
  Here is an example of a case that is not matched: Vuota
```

Che cosa portate a casa oggi

- Somma e prodotto si contano, e da lì viene il nome
- `option` è un tipo somma ordinario, e sposta un obbligo dal programmatore al compilatore
- La domanda giusta è **quali valori il tipo permette di scrivere**
- Parte della specifica si può spostare nel tipo — non tutta
- Un albero di operazioni è un programma, e a L11 lo eseguiremo

Al calcolatore

```
lezioni/L05-tipi-algebrici/codice/
  opzione.ml          togliete il caso None e leggete l'avviso
  somma_prodotto.ml   il conteggio, e lo stato illegale che compila
  somma_prodotto.cpp  struct, union e variant: chi è una somma
  albero.ml           la proprietà che tornerà a L08 e L09
  espressioni.ml      aggiungete Sottrazione e contate gli errori
  espressioni.cpp     togliete un case e contate le righe d'errore
  esercizi.ml         cinque esercizi, gli ultimi due senza codice
```

Riferimenti

- `codice/` — sette file, tutti eseguibili così come sono
- Esercizio da fare prima di L06: il punto 1, con la proprietà scritta **prima** dell'implementazione